

FRED TALKS

5th May 2026

CONTENTS

A non-anthropomorphized view of LLMs

HALVAR.FLAKE · 1478 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

Hidden Technical Debt of AI Systems: Agent Runtime

HAN, NOT SOLO · 3224 WORDS

A non-anthropomorphized view of LLMs

HALVAR.FLAKE · 14 SEP 2025 · [SOURCE](#)

In many discussions where questions of "alignment" or "AI safety" crop up, I am baffled by seriously intelligent people imbuing almost magical human-like powers to something that - in my mind - is just MatMul with interspersed nonlinearities.

In one of these discussions, somebody correctly called me out on the simplistic nature of this argument - "a brain is just some proteins and currents". I felt like I should explain my argument a bit more, because it feels less simplistic to me:

The space of words

The tokenization and embedding step maps individual words (or tokens) to some vectors. So let us imagine for a second that we have in front of us. A piece of text is then a path through this space - going from word to word to word, tracing a (possibly convoluted) line.

Imagine now that you label each of the "words" that form the path with a number: The last word with 1, counting forward until you hit the first word or the maximum context length . If you've ever played the game "Snake", picture something similar, but played in very high-dimensional space - you're moving forward through space with the tail getting truncated off.

The LLM takes your previous path into account, calculates probabilities for the next point to go to, and then makes a random pick into the next point according to these probabilities. An LLM instantiated with a fixed random seed is a mapping of the form .

In my mind, the paths generated by these mappings look a lot like strange attractors in dynamical systems - complicated, convoluted paths that are structured-ish.

Learning the mapping

We obtain this mapping by training it to mimic human text. For this, we use approximately all human writing we can obtain, plus corpora written by human experts on a particular topic, plus some automatically generated pieces of text in domains where we can automatically generate and validate them.

Paths to avoid

There are certain language sequences we wish to avoid - because the sequences these models generate try to mimic human speech in all its empirical structure, but we feel that some of the things that humans have empirically written are very undesirable to be generated. We also feel that a variety of other paths should ideally not be generated, if - when interpreted by either humans or other computer systems - undesirable results arise.

We can't specify strictly in a mathematical sense which paths we would prefer not to generate, but we can provide examples and counterexamples, and we try to hence nudge the complicated learnt distribution away from them.

"Alignment" for LLMs

Alignment and safety for LLMs mean that **we should be able to quantify and bound the probability with which certain undesirable sequences are generated**. The trouble is that we largely fail at describing "undesirable" except by example, which makes calculating bounds difficult.

For a given LLM (without random seed) and sequence, it is trivial to calculate the probability of the sequence to be generated. So if we had a way of somehow summing or integrating over these probabilities, we could say with certainty "this model will generate an undesirable sequence once every N model evaluations". We can't, currently, and that sucks, but at the heart, this is the mathematical and computational problem we'd need to solve.

The surprising utility of LLMs

LLMs solve a large number of problems that could previously not be solved algorithmically. NLP (as the field was a few years ago) has largely been solved.

I can write a request in plain English to summarize a document for me and put some key datapoints from the document in a structured JSON format, and modern models will just do that. I can ask a model to generate a children's book story involving raceboats and generate illustrations, and the model will generate something that is passable. And much more, all of which would have seemed like absolute science fiction 5-6 years ago.

We're on a pretty steep improvement curve, so I expect the number of currently-intractable problems that these models can solve to keep increasing for a while.

Where anthropomorphization loses me

The moment that people ascribe properties such as "consciousness" or "ethics" or "values" or "morals" to these learnt mappings is where I tend to get lost. We are speaking about a big recurrence equation that produces a new word, and that stops producing words if we don't crank the shaft.

To me, wondering if this contraption will "wake up" is similarly bewildering as if I was to ask a computational meteorologist if he isn't afraid of his meteorological numerical calculation will "wake up".

I am baffled that the AI discussions seem to never move away from treating a function to generate sequences of words as something that resembles a human. Statements such as "an AI agent could become an insider threat so it needs monitoring" are simultaneously unsurprising (you have a randomized sequence generator fed into your shell, literally **anything** can happen!) and baffling (you talk as if you believe the dice you play with had a mind of their own and could decide to conspire against you).

Instead of saying "we cannot ensure that no harmful sequences will be generated by our function, partially because we don't know how to specify and enumerate harmful sequences", we talk about "behaviors", "ethical constraints", and "harmful actions in pursuit of their goals". All of these are anthropocentric concepts that - in my mind - do not apply to functions or other mathematical objects. And using them muddles the discussion, and our thinking about what we're doing when we create, analyze, deploy and monitor LLMs.

This muddles the public discussion. We have many historical examples of humanity ascribing bad random events to "the wrath of god(s)" (earthquakes, famines, etc.), "evil spirits" and so forth. The fact that intelligent highly educated researchers talk about these mathematical objects in anthropomorphic terms makes the technology seem mysterious, scary, and magical.

We should think in terms of "this is a function to generate sequences" and "by providing prefixes we can steer the sequence generation around in the space of words and change the probabilities for output sequences". And for every possible undesirable output sequence of a length smaller than , we can pick a context that maximizes the probability of this undesirable output sequence.

A much clearer formulation, which helps more clearly articulate the problems to solve.

Why many AI luminaries tend to anthropomorphize

Perhaps I am fighting windmills, or rather a self-selection bias: A fair number of current AI luminaries have self-selected by their belief that they might be the ones getting to AGI - "creating a god" so to speak, the creation of something like life, as good as or better than humans. You are more likely to choose this career path if you believe that it is feasible, and that current approaches might get you there. Possibly I am asking people to "please let go of the belief that you based your life around" when I am asking for an end to anthropomorphization of LLMs, which won't fly.

Why I think human consciousness isn't comparable to an LLM

The following is uncomfortably philosophical, but: In my worldview, humans are dramatically different things than a function. For hundreds of millions of years, nature generated new versions, and only a small number of these versions survived. Human thought is a poorly-understood process, involving enormously many neurons, extremely high-bandwidth input, an extremely complicated cocktail of hormones, constant monitoring of energy levels, and millions of years of harsh selection pressure.

We understand essentially nothing about it. In contrast to an LLM, given a human and a sequence of words, I cannot begin putting a probability on "will this human generate this sequence".

To repeat myself: To me, considering that any human concept such as ethics, will to survive, or fear, apply to an LLM appears similarly strange as if we were discussing the feelings of a numerical meteorology simulation.

The real issues

The function class represented by modern LLMs are very useful. Even if we never get anywhere close to AGI and just deploy the current state of technology everywhere where it might be useful, we will get a dramatically different world. LLMs might end up being similarly impactful as electrification.

My grandfather lived from 1904 to 1981, a period which encompassed moving from gas lamps to electric, the replacement of horse carriages by cars, nuclear power, transistors, all the way to computers. It also spanned two world wars, the rise of Communism and Stalinism, almost the entire lifetime of the USSR and GDR etc. The world on his birth looked nothing like the world when he died.

Navigating the dramatic changes of the next few decades while trying to avoid world wars and murderous ideologies is difficult enough without muddying our thinking.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · [SOURCE](#)

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding** != **vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments...). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs... but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name “delete-everything”. Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

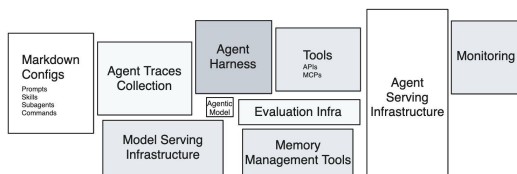
In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

Hidden Technical Debt of AI Systems: Agent Runtime

HAN, NOT SOLO · 27 APR 2026 · [SOURCE](#)

Eleven years ago, Sculley et al. drew the diagram everyone in MLOps has seen: a tiny black box labeled “ML Code” surrounded by a sprawl of much larger boxes — data collection, feature extraction, configuration, monitoring, serving infrastructure. The point of the diagram was that the model code is the smallest piece of a real ML system, and that everything else is where the technical debt accumulates.



Hidden Technical Debt in Agentic AI Systems

The same diagram is being redrawn for agents. The agentic model call is the small box. The largest box to the right that is currently driving most of the spend and influencing how system architecture will be done in the future incidents, is the **agent runtime**, or agent serving infrastructure.

The agent is not the model. The agent is the harness plus the model, running inside the runtime. And almost nobody is treating it that way.

What an Agent Runtime Actually Is

A agentic model by itself maps from text input to text output. Other modalities are in play, but the primary fabric is still text. An agent is what you get when you wrap the agentic model with harnesses so it can take actions, observe effects, feed those observation back into the next call. The execution environment where that happens is the runtime.

Concretely, an agent runtime is the union of:

- **compute substrate** — a container, microVM, or full VM where code runs.
- **filesystem** the agent can read and write, often with snapshot and rollback semantics.
- **tools** — shell, code interpreter, browser, file editor, MCP servers — exposed to the model as callable interfaces.
- **network boundary** that defines what the agent can reach and what can reach it.
- **state model** that decides what persists across turns, across episodes, and across users.

- **lifecycle controller** that starts, suspends, snapshots, resumes, and tears down environments.

In the taxonomy of RL environments for LLM agents, these are the (harness) and (state) components. In production, this is the part that decides whether your agent finishes a task in eight seconds or eight minutes, whether two users can share an environment safely, and whether a malicious prompt can read your secrets.

Most teams shipping agents today have not built this layer. They have rented it, glued it together from cloud primitives that were designed for a different workload, or simply not thought about it yet. That last group is the one accumulating the most debt.

Why Sandboxing Is Not Optional

Models hallucinate code. They run `rm -rf`. They paste credentials into curl commands. They follow instructions embedded in untrusted documents. They retry the same failing command in a tight loop and burn through a quota. None of these are exotic edge cases — they are the modal behavior of capable models when handed unrestricted tool access.

The sandbox is what stops these behaviors from becoming incidents. Four reasons it has to exist as a first-class layer rather than a hopeful disclaimer in the system prompt:

Isolation against the model's mistakes. A coding agent that mounts your repo and has shell access can, and eventually will, delete the wrong directory. Filesystem isolation, copy-on-write snapshots, and per-session ephemerality turn destructive actions into recoverable ones.

Isolation against prompt injection. The agent reads tool outputs. Tool outputs are attacker-controlled the moment the agent visits a webpage, opens a PDF, or processes a customer email. A sandbox is the only thing standing between an injected instruction and your production database. This is the agent-systems version of the SQL injection lesson, and we are at roughly the 2003 stage of learning it.

Multi-tenancy at training scale. RL training spins up thousands of concurrent rollouts. Each rollout needs its own filesystem, its own process tree, its own network namespace. Without strong isolation primitives, one rollout's flaky shell command takes down a neighbor's training step.

Reproducibility. A sandbox you can snapshot is a sandbox you can replay. Replay is how you debug a six-hour agent trajectory without re-running the whole thing. It is also how you turn a production failure into a regression test.

A web app’s runtime can be sloppy because the user is the one driving and a refresh fixes most things. An agent’s runtime cannot, because the agent will keep going long after a human would have stopped to ask a question.

The Cognition team is direct about this in [their post-mortem on building Devin’s cloud agent infrastructure](#): containerized agents share a kernel, and a single compromised session can reach every other container’s filesystem, credentials, and network connections. Because agents generate their own code, run arbitrary commands, and probe the environment in unpredictable ways, kernel-escape is not a theoretical risk — it is the working assumption. Their conclusion, after more than a year of hypervisor engineering, is the same one the broader infrastructure community has converged on for any untrusted-code workload: VM-level isolation, where each session gets its own kernel and there is no shared attack surface.

Manus team has famously built and snapped shotted all their agent runtimes in a restorable format so users can come back in the future to replay what agents have done.

The Isolation Primitive Stack

Most of the differentiation between sandbox vendors lives one layer down, in the isolation primitive they pick. There are five primitives in serious use, with very different trade-offs.

Primitive	Isolation model	Cold start	Workload fit	Notable users
Linux containers (runc, Podman)	Shared host kernel, namespaces + cgroups + seccomp	~100ms	Trusted code, internal CI	Docker, Kubernetes default
Firecracker	KVM-based microVM, dedicated kernel per VM, ~5MB Rust VMM	~125ms boot, sub-second from snapshot	Untrusted code at high density	AWS Lambda, Fargate, E2B, Fly.io
gVisor	Userspace kernel intercepting syscalls; runc-compatible runtime	Container-class	Defense in depth where a microVM is overkill; GPU virtualization	Google Cloud Run, App Engine, Modal
Kata Containers	Lightweight VM per pod, OCI-compatible	Few hundred ms	Multi-tenant Kubernetes	Confidential Containers, some managed K8s
V8 isolates				

Per-tenant JS heap inside a single process	Sub- millisecond	JavaScript-only, no arbitrary binaries	Cloudflare Workers, Deno Deploy
--	---------------------	---	---------------------------------------

A few things are worth saying out loud about this table.

Containers are not a sandbox for agent code. They are a packaging and resource-control mechanism. The shared kernel means a kernel exploit, a misconfigured capability, or a sloppy seccomp profile takes down the isolation boundary. It is not okay for an agent that may execute attacker-controlled instructions hidden in a web page.

Firecracker is the de facto industry primitive for this category. AWS open-sourced it in 2018 to power Lambda and Fargate, and almost all agent-sandbox startup runs on top of it, including E2B, Fly.io, Vercel Sandbox. It boots a stripped-down Linux kernel inside KVM in roughly 125 milliseconds and uses about five megabytes of memory for the VMM itself. The combination of strong isolation, fast boot, high density is what makes thousands of concurrent rollouts economically viable.

gVisor is the middle path. It runs a userspace kernel that intercepts and re-implements Linux syscalls, giving you stronger isolation than namespaces without paying the full cost of a hardware VM. The trade-off is that some syscalls are slower or unsupported, which matters for workloads that hit the kernel hard. Google uses it for Cloud Run and App Engine; it is the right pick when you want defense-in-depth on top of containers but cannot move to microVMs.

Kata Containers fit a specific niche. They are an OCI-compatible runtime that wraps each pod in a lightweight VM, which lets you run untrusted workloads on Kubernetes without rewriting the orchestration layer. The cost is that you inherit Kubernetes’ assumptions about pod lifetime, which do not match how agents actually behave.

V8 isolates are the wrong primitive for general agent workloads. They are extraordinary for JavaScript-only edge compute but an agent that needs to run arbitrary Python, install packages, drive a browser, or execute compiled binaries cannot live inside a V8 heap. Cloudflare’s own Sandbox product addresses this by adding container-class compute alongside isolates.

The picks higher up the stack inherit these trade-offs. When a sandbox vendor advertises “VM-level isolation” they almost always mean Firecracker. When they advertise “millisecond cold starts” without VM isolation, they usually mean V8 isolates with a different threat model. The vendor brochure abstracts the primitive away; the threat model and the workload do not.

The Startup Sandbox Landscape

A category of “sandbox-as-a-service” companies has emerged in the last two years, almost all of them building on top of the primitives above. [Northflank’s roundup of Modal Sandboxes alternatives](#) is a good market snapshot — the differentiation is in the developer ergonomics, the snapshot model, and the mix of tools that come pre-wired, not in the isolation layer underneath.

Vendor	Isolation	Snapshot model	Cold start	Notable design choice
Modal	gVisor	Filesystem diffs from base image	Sub-second from snapshot	GPU support
E2B	Firecracker microVM	Full VM snapshot	~150ms	Open-source SDK, language-agnostic, popular in the agent dev community
Daytona	Containers / VMs	Forkable workspaces	Few seconds	Forked dev environments, OCI-compatible
Northflank	Containers, GPU-aware	Persistent volumes	Container-class	GPU sandboxes for agents that train or run inference inside the loop
Browserbase / Steel / Hyperbrowser	Containerized browsers	Session replay	Seconds	Browser-only runtimes for web agents — DOM, screenshots, CDP
Cloudflare Workers Sandbox	V8 isolates + containers	Object snapshots	Milliseconds	Edge-first, shared-nothing model
Vercel Sandbox	Firecracker	Snapshot from build	Sub-second	Tied to Vercel’s deploy and preview model

The reference Modal case study is [Ramp Inspect](#): a background coding agent that now writes more than half of Ramp’s merged pull requests, with each session running in its own Modal sandbox containing Postgres, Redis, Temporal, RabbitMQ, a VS Code server, and a VNC stack with Chromium. Filesystem snapshots are refreshed every thirty minutes by a cron, so a new session is working on a prompt within seconds. The lesson buried in that architecture is that the agent’s productivity is bounded by the runtime’s startup time, not by the model’s tokens-per-second.

The browser-runtime category exists because driving Chromium is its own engineering problem — CDP, profiles, residential IPs, captcha handling, session persistence — and no general-purpose sandbox has it built in. Expect this to consolidate into the general-purpose vendors over the next eighteen months, the same way headless browsers absorbed into the major test frameworks.

It is also worth noting what nobody on this list does themselves. None of these vendors writes their own hypervisor. They all stand on top of the primitives in the previous section, which is what lets the category exist. The startups that have tried to build a complete stack — including the hypervisor — have ended up looking more like Cognition: years of engineering investment to get something that is finally indistinguishable from “we run on Firecracker with custom snapshot logic.”

The Hyperscaler Sandbox Landscape

The hyperscalers got to this market late and are still catching up.

AWS Bedrock AgentCore ships a Code Interpreter and a Browser Tool as managed runtimes for agents. The isolation is microVM-class, the integrations point at the rest of the AWS data plane, and the pricing model is per-session. However, it has major gaps relative to the startups, where the runtime has to be coded and shipped like a Lambda function, instead of having the flexibility to support heterogeneous workloads, e.g. different agent harness + model combinations. Not a major problem for production, but a major problem for research and development.

Azure Container Apps Dynamic Sessions uses Hyper-V isolation and advertises sub-second start times for code interpreter sessions. It is the most directly comparable hyperscaler offering to E2B and Modal in terms of intent. The integration story is strongest if you are already on Azure OpenAI and AKS. It also suffers from the flexibility to support heterogeneous workloads.

GCP Cloud Run Sandboxes is ahead of the pack with the usability similar to Modal, E2B, Daytona and the likes. It's built on top of Cloud Run, which uses gVisor. Supports heterogeneous workloads.

Lambda, Fargate, App Runner, Cloud Run. It is tempting to use these as agent sandboxes because they are cheap and already in your account. They are designed for stateless, request-response workloads with strict execution-time limits and no native snapshot model. They will work for a demo. They will not work for an agent that runs for six hours, mutates a filesystem, and needs to fork mid-trajectory.

The hyperscaler offerings are converging on the right shape — microVM isolation, fast snapshots, managed tools — but the primitives underneath were built for web workloads. The startups had the advantage of building on top of those primitives without inheriting the legacy product surface area.

Experimentation and Production Want Different Runtimes

One of the major difference how AI engineering differs from traditional web development is the difference in development and production infrastructure requirements. Here's the difference between your training cluster (if you are training models), experimentation and evaluation runs, vs production workload.

Dimension	Experimentation / Training	Production / Serving
Concurrency shape	Thousands of parallel rollouts, bursty	One session per user, steady-state
Cold start tolerance	Critical — 5s × 10k rollouts is real money	Forgivable — users wait
State model	Fork, branch, replay, snapshot	Durable, per-user, auditable
Network policy	Often offline or recorded for determinism	Open internet, real APIs
Failure model	Drop the rollout, sample more	Retry, degrade, page someone
Observability	Full trace, every token, replayable	SLOs, error budgets, sampling
Cost model	Spot, preemptible, batch	Reserved, predictable, latency-bound
Determinism	Required for reproducible runs	Often counterproductive
Lifetime	Seconds to minutes	Minutes to hours, or pinned for a session

A training rollout wants to start in 200 milliseconds, run for thirty seconds against a frozen snapshot of the world, get scored, and disappear. A production session wants to start in two seconds, hold state for a forty-minute conversation, talk to the live internet, and survive a transient failure without losing the user's work.

The production side has a requirement that almost nobody surfaces in eval reports: the agent needs to survive the **async gaps** in real engineering work. [Cognition's experience report](#) is the cleanest articulation of this — an agent opens a PR, waits on CI, responds to a review comment, reruns tests, pushes a follow-up commit. Between each step there are minutes, hours, sometimes

days where the agent’s working state has to persist. A containerized agent can only survive these gaps by burning compute to stay alive, and if the container is rescheduled or times out the session is lost. Their solution was hypervisor-level snapshotting of the full machine state — memory, process tree, filesystem — so compute shuts down while the agent is idle and resumes exactly where it left off when a CI result arrives. Building that took longer than any other piece of infrastructure they had shipped to date.

Optimizing the same runtime for both training and production is how you end up with a system that is too slow for training and too brittle for production. Most teams discover this after the first time they try to “just deploy what we trained on” and find the latency budget eaten by a startup model that was fine when amortized over ten thousand rollouts and intolerable when paid by a single user staring at a spinner.

Dev/Prod Parity Is the Real Problem

In 2011, [Twelve-Factor App](#) told us to keep development, staging, and production as similar as possible. That advice was about reducing the gap between where the engineer types code and where the code runs. For agents, the gap that matters is between where the agent **trained** and **experimented** and where the agent **runs**.

An agent learns the runtime. Tool latencies, failure modes, shell quirks, filesystem layout, the exact way `ls` formats output, the way the browser resolves redirects. The model picks up all of it during training and bakes it into the policy, or the optimizer (RLM or GEPA) picks up all of it during training and bakes it into the harness or the prompts. Move these to a different runtime and the behavior shifts. Tools that were idempotent become flaky. Commands that were instant now block. Snapshots that existed disappear. The agent has no abstract concept of “shell”; it has a concept of “the shell I trained on.”

This is a new flavor of distributional shift. It is not the data shift the MLOps community has been worrying about for a decade. It is **runtime shift**, and it shows up as silent quality regressions that no eval catches because the eval runs in the training runtime.

There are three honest paths through this:

Co-locate train and prod on the same runtime. Pick one sandbox provider, run RL rollouts on it, run production sessions on it, accept the lock-in. This is what the most disciplined AI engineering teams are doing.

Define a runtime contract and enforce it on both sides. A small, versioned interface — “this is the shell, these are the tools, these are the latencies, these are the failure modes” — that you implement once on your training infrastructure and once on your production infrastructure. The contract is the abstraction; the sandbox underneath can vary. This is harder than it sounds because the contract has to cover not just the API surface but the timing and failure semantics that the agent has learned.

Train against production noise. Inject latency, errors, and tool failures during training so the agent’s policy is robust to runtime variance instead of dependent on a specific runtime. Step-DeepResearch reports tangible gains from injecting 5–10% tool errors during training. This is the runtime analog of dropout.

The wrong answer, the one most teams are unconsciously picking, is to pick a sandbox for production as a software engineering solution, and forget all about the requirements from AI and machine learning, then spend the next quarters chasing flakiness of agent performances.

The Bill That Is Coming Due

Sculley’s argument was that ML systems accumulate technical debt in places that traditional software engineering does not have language for — feedback loops, configuration sprawl, undeclared consumers, glue code. Agent systems inherit all of that and add a new line item: the runtime debt.

Runtime debt looks like this. A team picks the sandbox that was easiest to integrate at prototype time. Production traffic exposes a different mix of failure modes. The team patches around the gap with retries, longer timeouts, and prompt-engineered apologies. The agent’s behavior gets entangled with that runtime’s quirks with whack-a-mole without disciplined research and development. A year later, switching runtimes is a six-month migration because nobody can predict which behaviors will move.

The agent has to live on a runtime. The teams that internalize that early will spend the next two years compounding the advantage. The teams that do not will spend the next two years paying down a debt they did not know they were taking on.

References

- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. NeurIPS. https://papers.nips.cc/paper_files/paper/2015/hash/86df7dcfd896fcacf2674f757a2463eba-Abstract.html
- Lee, H. (2026). A Taxonomy of RL Environments for LLM Agents. Han, Not Solo. <https://leehanchung.github.io/blogs/2026/03/21/rl-environments-for-llm-agents/>
- Workman, G. (2026). How Ramp built a full-context background coding agent on Modal. Modal Blog. <https://modal.com/blog/how-ramp-built-a-full-context-background-coding-agent-on-modal>
- Lopopolo, R. (2026). Harness engineering: leveraging Codex in an agent-first world. OpenAI. <https://openai.com/index/harness-engineering/>
- Microsoft. (2024). Azure Container Apps Dynamic Sessions. <https://learn.microsoft.com/en-us/azure/container-apps/sessions>
- Firecracker. AWS open-source microVM. <https://firecracker-microvm.github.io/>
- Northflank. (2026). Top Modal Sandboxes Alternatives for Secure AI Code Execution. <https://northflank.com/blog/top-modal-sandboxes-alternatives-for-secure-ai-code-execution>

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {Hidden Technical Debt of AI Systems: Agent Runtime},
  year = {2026},
  month = {04},
  day = {24},
  howpublished = {\url{https://leehanchung.github.io}},
  url = {https://leehanchung.github.io/blogs/2026/04/24/hidden-tech
}
```